

Algorithm–hardware co-design of neuromorphic networks with dual memory pathways

Received: 8 December 2025

Accepted: 11 May 2026

Published online: 16 June 2026

 Check for updates

Pengfei Sun ^{1,6}✉, Zhe Su ^{2,6}✉, Jascha Achterberg³, Giacomo Indiveri², Dan F. M. Goodman ¹ & Danyal Akarca ^{1,4,5}✉

Spiking neural networks excel at event-driven sensing. Yet, maintaining task-relevant context over long timescales both algorithmically and in hardware, while respecting both tight energy and memory budgets, remains a core challenge in the field. Here we address this challenge through an algorithm–hardware co-design effort. At the algorithm level, inspired by the cortical fast–slow organization in the brain, we introduce a neural network with an explicit slow memory pathway that, combined with fast spiking activity, enables a dual memory pathway architecture in which each layer maintains a compact low-dimensional state that summarizes recent activity and modulates spiking dynamics. This explicit memory stabilizes learning while preserving event-driven sparsity, achieving competitive accuracy on long-sequence benchmarks with 40–60% fewer parameters than equivalent state-of-the-art spiking neural networks. At the hardware level, we introduce a near-memory-compute architecture that fully leverages the advantages of the dual memory pathway architecture by retaining its compact shared state while optimizing data flow, across heterogeneous sparse-spike and dense-memory pathways. We show experimental results that demonstrate more than a fourfold increase in throughput and over a fivefold improvement in energy efficiency compared with state-of-the-art implementations. Together, these contributions demonstrate that biological principles can guide functional abstractions that are both algorithmically effective and hardware-efficient, establishing a scalable co-design framework for real-time neuromorphic computation and learning.

Brains solve temporal problems in real time and under tight energy budgets, and are unmatched by artificial systems. One hypothesis for the brain's superior temporal processing abilities is that their computation is both continuous time and event-driven, based on discrete sparse spiking. Spiking neural networks (SNNs) inherit these two core principles, making them a promising and biologically grounded complement to dense artificial neural networks^{1,2}. Yet one brain-like ability critically important for temporal processing remains hard to reproduce efficiently: retaining information over behaviourally relevant timescales. Standard

leaky integrate-and-fire (LIF) neurons integrate spikes into a decaying membrane potential³, which captures evidence instantaneously but progressively suppresses older inputs over millisecond timescales^{4,5}. Conversely, precise temporal codes (for example, time-to-first-spike) preserve timing but do not carry graded intensity over long windows^{6–8}. Another way of thinking about this is that a purely feedforward SNN (FSNN) typically captures either what happened or when it happened, but not both. The fundamental challenge is therefore maintaining long-range temporal context without sacrificing event-driven efficiency.

A full list of affiliations appears at the end of the paper. ✉e-mail: p.sun@imperial.ac.uk; zhesu@ini.ethz.ch; d.akarca@imperial.ac.uk

Several strategies have been explored to extend temporal capacity in SNNs⁹. One approach augments recurrence, often via heterogeneous time constants or adaptive thresholds^{10–14}, allowing activity to persist through feedback. Another exploits biologically inspired transmission delays (axonal, synaptic or dendritic) to align spikes to task-relevant timescales and act as implicit temporal buffers^{15–25}. Both strategies improve temporal integration, yet each imposes substantial implementation costs in hardware. Dense recurrence scales quadratically with layer width, erodes sparsity benefits, and requires continuous recirculation of activity, which increases both the memory footprint and energy consumption. Long, learnable delays avoid explicit recurrence but can demand deep on-chip buffers and per-connection timing metadata, inflating area, power and latency in neuromorphic implementations^{26,27}. We also note that reservoir-based spiking approaches—exemplified by liquid-state machines—exploit rich transient dynamics in a fixed recurrent substrate and learn only a simple readout, offering an alternative route to temporal processing²⁸. However, in more challenging real-world settings their accuracy can be limited, particularly under constrained hardware deployment^{29,30}. This creates a design challenge: temporal memory mechanisms must be sufficiently expressive to capture long-range dependencies while remaining compatible with efficient neuromorphic implementation. Addressing this requires reconsidering both algorithmic structure and hardware design^{2,31–36}.

The mammalian cortex inspires a potential architectural solution. Cortical circuits maintain efficient long-range temporal context through dendritic branches that integrate inputs over a spectrum of timescales, while neuronal populations have correspondingly heterogeneous temporal constants that shape fast somatic spiking without requiring dense global recurrence^{24,37,38}. Recent evidence suggests that cortical networks dynamically recruit fast and slow computational pathways according to task demands, revealing a functional organization built around multiple interacting timescales^{39,40}. We distill this into a functional abstraction: pair fast spiking dynamics with compact temporal memory. Following this abstraction, we introduce a memory-augmented spiking architecture in which each layer maintains a low-dimensional state vector $m \in \mathbb{R}^d$, with $d \ll N$ for a layer of N spiking neurons. This state evolves under well-conditioned slow dynamics and feeds back as an additional input current to the neurons, acting as a compact linear slow pathway that provides shared context and modulates fast spiking within each layer. This deployment-oriented fast–slow decomposition enables controllable trade-offs between memory footprint, and accuracy under tight resource budgets. Rather than storing full spike histories or broadcasting dense recurrent activity, the network compresses recent activity into a few slow modes and exposes that compressed context at each time step. This dual memory pathway (DMP) architecture is motivated by long-horizon temporal memory under stringent hardware constraints, and is thus tailored to regimes in which long-range temporal dependencies are the primary limiting factor.

To fully exploit the advantages afforded by this memory architecture, we introduce a digital near-memory-compute architecture that uses heterogeneous data-flow optimization to maximize arithmetic intensity. This design overcomes limitations of previous hardware, in which conventional recurrent SNN accelerators must scale both parameter storage and computation quadratically⁴¹, while hardware supporting long learnable delays requires deep buffers in digital implementations⁴² or large capacitors in mixed-signal designs²³, both significantly increasing energy consumption and chip area. The algorithmic structure we propose inherently enables a low-memory-footprint design that avoids these costs. Through algorithm–hardware co-design, we empirically observe that our system matches or outperforms recurrent and delay-based SNNs on event-driven auditory classification and long-horizon sequential inference, all while using only a small number of memory states. Post-layout simulations of the proposed hardware architecture in an advanced 22FDX technology

demonstrate more than $4 \times$ higher throughput and over $5 \times$ better energy efficiency compared with state-of-the-art (SOTA) designs.

Overall, we show how efficient temporal processing can be achieved through hardware–algorithm co-design inspired by circuit motifs in the brain. Imposing hardware considerations at the algorithmic level results in a network architecture with a temporal memory that is explicit and low-rank. By designing hardware around explicit memory and applying carefully orchestrated optimizations, we obtain a heterogeneous data-flow structure that efficiently integrates fast and slow memory. Jointly, this algorithm and hardware pairing achieves benchmark-beating temporal processing capabilities alongside SOTA energetic efficiency. We made the hardware architecture open source to enable community adoption and further co-design exploration.

Results

Dual memory pathways for efficient temporal computation

Feedforward spiking networks are computationally efficient but perform poorly on tasks requiring long temporal dependencies, as their leaky membranes encode only transient evidence and rapidly forget past inputs. Recurrent SNNs (RSNN) extend temporal integration by feeding activity back within each layer, enabling longer retention but at the cost of quadratic scaling and high memory traffic that erode event-driven efficiency. Recent delay-based SNNs (DSNN), particularly those with learnable axonal delays, address this limitation by distributing memory across time, achieving remarkable performance on neuromorphic benchmarks with only linear parameter complexity ($\mathcal{O}(N)$)⁴³. However, these models rely on deep programmable buffers, which can increase hardware cost and latency. Here we introduce the DMP architecture for spiking neural networks (DMP-SNN), which replaces these costly mechanisms with a compact, shared slow state that summarizes recent activity and feeds back as an additional input current (Fig. 1a). Acting as an explicit low-dimensional working memory, at only 5% of the hidden width, this state preserves the temporal context relevant to the task over behaviourally meaningful timescales while remaining fully compatible with efficient hardware implementation (Fig. 1b).

We evaluated this DMP architecture on two classes of sequential benchmarks. The first class targets long-range visual integration. In sequential MNIST (S-MNIST), each 28×28 image is presented as a 784-step pixel stream, requiring integration across hundreds of steps. In permuted sequential MNIST (PS-MNIST), a fixed random permutation destroys local spatial structure, so performance depends almost entirely on long-range temporal memory⁴⁴. The second class targets event-based auditory classification⁴⁵. The Spiking Heidelberg Digits (SHD) dataset encodes spoken digits as 700-channel spike trains generated by a cochlear model, and the Spiking Speech Commands (SSC) dataset similarly represents 35 spoken-word categories. Together, these benchmarks span dense, clocked sequences and irregular, event-driven input, and probe temporal dependencies on different timescales.

We benchmarked four spiking network architectures: FSNN, RSNN, DSNN and our DMP-SNN (Table 1). Feedforward SNNs fail on long-range structure tasks (11.3% on PS-MNIST). Recurrence in RSNNs improves temporal integration (71.0% on PS-MNIST; 71.4% on SHD) but scales as $\mathcal{O}(N^2)$. Delay SNNs perform well on auditory tasks (91% on SHD; 70% on SSC) by learning axonal timing, but require long programmable delays. The DMP-SNN reaches 99.3% on S-MNIST and 97.3% on PS-MNIST. Our method matches delay-based models on SHD and SSC, and outperforms them in the single-layer SSC setting. Figure 2a shows that even a small number of memory states yields consistent gains, and that once the state buffer length exceeds roughly 25% of the sequence, performance is already competitive, indicating that the relevant temporal information has been captured. Figure 2b illustrates the learning dynamics of the baseline model relative to the DMP-SNN, showing that memory-augmentation leads to both quicker convergence and higher accuracy. Crucially, this is achieved using only a low-dimensional auxiliary state with $\mathcal{O}(d)$ overhead, where $d \ll N$ (typically 5–10% of the

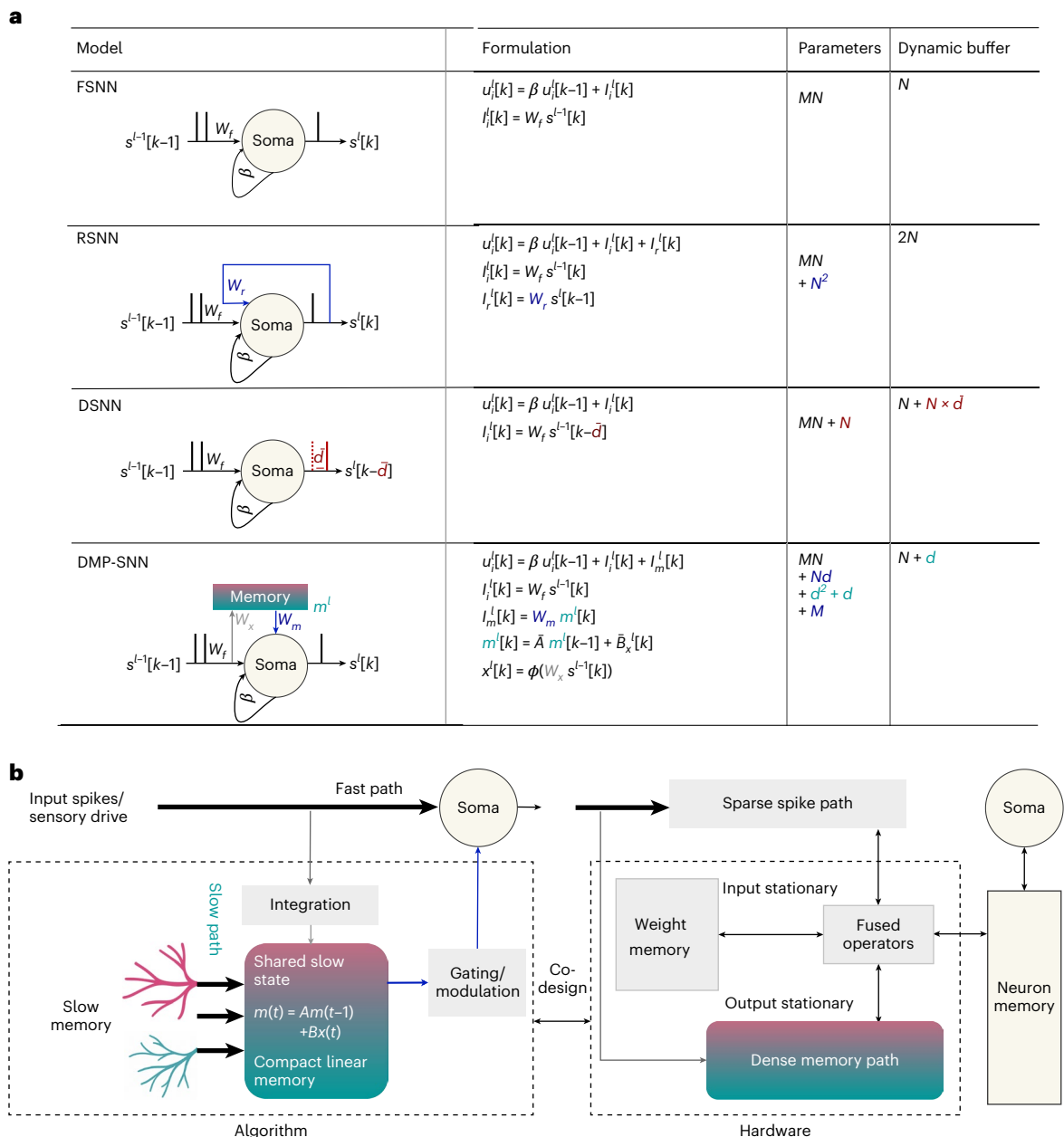


Fig. 1 | From fast–slow cortical motifs to the DMP architecture. **a**, Schematic comparison of FSNN, RSNN, DSNN and the proposed DMP-SNN, highlighting their discrete-time formulations, parameter counts and dynamic buffer requirements. M , input dimension; N , output dimension; d , memory dimension ($d \ll N$); $\tilde{d}$, axonal delay. **b**, Dual memory pathway abstraction. At the algorithmic

level, each layer maintains a shared, low-dimensional state that captures slow contextual dynamics and modulates fast spiking activity. At the hardware level, this separation is mirrored by a heterogeneous accelerator that keeps the compact state on-chip and fuses sparse and dense computations for efficient execution.

hidden width) can match or exceed SOTA performance with fewer parameters while providing long-timescale context (see solution I, the most parameter-efficient solution, and solution II, the best-performing solution, in Fig. 2c)^{13,14,24,46–49}. Extended Data Table 1 further reports results on the DVS Gesture spatiotemporal event-stream benchmark, where incorporating DMP yields a consistent accuracy improvement with negligible parameter overhead.

Context-dependent temporal demands reveal design trade-offs
We next examined how frequently the auxiliary memory must be updated within the dual pathway architecture. To do this, we imposed temporal sparsity by updating the membrane potential over fixed numbers of steps. We found that sequence-vision tasks (S-MNIST,

PS-MNIST) were largely unaffected even at a skip length (dilation) of ten, whereas auditory tasks (SHD, SSC) degraded much sooner. Regular dense inputs therefore tolerate coarse updates, whereas irregular spike streams require tighter coupling (Fig. 3a), which, foreshadowing our later results, is favourable for hardware since less frequent updates lower the switching activity and memory traffic with little loss on long-horizon vision benchmarks. To make this trend explicit, Extended Data Fig. 1 summarizes, for each memory-state size, the competitive configurations, showing that strong performance tends to favour relatively short buffers.

Recent spiking state-space architectures have attempted to remove the feedforward drive (fast pathway) entirely, compensating by using residual connections and additional mapping layers

Table 1 | Performance on temporally structured benchmarks

Model	Parameters	Accuracy (%)
PS-MNIST		
FSNN	42,000	11.30
RSNN	122,000	71.00
DSNN	43,000	72.06
DMP-SNN (I)	61,000	95.50
DMP-SNN (II)	102,000	96.65
DMP-SNN (peak accuracy)	202,000	97.32
S-MNIST		
FSNN	42,000	59.00
RSNN	122,000	74.00
DSNN	43,000	88.79
DMP-SNN (I)	51,000	98.08
DMP-SNN (II)	73,000	99.20
DMP-SNN (peak accuracy)	202,000	99.28
SHD		
FSNN	37,000	48.60
RSNN	70,000	71.40
DSNN	37,000	90.98
DMP-SNN (I)	38,000	89.00
DMP-SNN (II)	40,000	91.23
DMP-SNN (peak accuracy)	46,000	91.69
SSC		
FSNN	22,000/39,000	26.08/38.50
RSNN	39,000/72,000	50.90/60.00
DSNN	23,000/39,000	60.01/69.40
DMP-SNN (I)	23,000/40,000	65.09/69.50
DMP-SNN (II)	24,000/42,000	65.37/72.90
DMP-SNN (peak accuracy)	24,000/42,000	65.37/72.90

Chance level: 10.00 (S-MNIST or PS-MNIST), 5.00 (SHD), 2.90 (SSC).

Test accuracy (%) on PSMNIST, SMNIST, SHD and SSC for four model families: FSNN, RSNN, DSNN and the proposed DMP-SNN. For SSC, results are shown for networks with one and two hidden layers (one- or two-layer). Beige rows mark DMP-SNN in three settings: solution I (the most parameter-efficient, matching strong baselines); solution II (accuracy–efficiency trade-off, achieving high accuracy and maintaining low parameter cost); and peak accuracy. Chance-level accuracy is shown for reference. Bold font indicates this work.

(Extended Data Fig. 2). Although this approach is also effective, it substantially increases model complexity—often by up to threefold in parameters—and imposes further residual computations that raise hardware cost^{50–53}. In contrast, removing the feedforward pathway in our model caused accuracy to drop to chance across all datasets, irrespective of memory size. Extended Data Table 2 further reports the accuracy and parameter counts of these model variants, highlighting the improved accuracy–efficiency balance achieved by DMP relative to both the vanilla state-space model and its spiking counterpart. This indicates that the slow memory state alone cannot sustain task performance. The explicit temporal state instead serves as a lightweight contextual modulator that reinforces stimulus-driven spiking rather than acting as a standalone recurrent mechanism. Consistent with this, model capacity on the SHD dataset improved with larger hidden width and memory dimension until task-relevant timescales were captured, beyond which gains saturated (Fig. 3b). These results suggest that neuron count, memory size and temporal horizon should be co-tuned to achieve balanced temporal modelling without excessive computational or hardware overhead.

We next hypothesized that the explicit state already carries long-range context and therefore reduce the need for long axonal delays. Delays are known to align spikes to task-relevant timescales and to improve neuromorphic speech tasks, but long delays require deep buffers that scale with the maximum delay and specialized timing support^{54,55}. Consistent with our hypothesis, we found that combining heterogeneous axonal delays with our memory-augmented neurons improved accuracy for small memory states and, importantly, shifted the learned delays towards shorter horizons. With memory dimension $d = 5$ and a maximum delay window $\theta = 5$, accuracy increased by 2% despite the tight delay budget, and enlarging the delay window mainly accelerated optimization rather than being strictly required (Fig. 3c,d). The learned axonal delay distributions are shown in Extended Data Fig. 3.

Together, this points to a hardware-favourable hybrid solution in which the shared temporal state supplies the long-range component without per-connection buffering, whereas short delays provide fine timing alignment. In this regime, long programmable delays become optional rather than structural, and the network retains the benefits of delay coding, including robustness to low-precision weights and discretisation⁴³, and fast motion detection⁵⁶, while easing mapping to heterogeneous neuromorphic hardware.

Dual memory networks enhance long-term credit assignment

To assess whether the proposed architecture also facilitates long-range credit assignment, we used last-time step supervision and quantified how the loss at the final step back-propagates to the first layer over time. In this regime the network must retain task-relevant information across the entire sequence. As shown in Extended Data Fig. 4, gradients in the first layers remained non-negligible over the full 784 steps of S-MNIST and over 200 steps of PS-MNIST, while accuracies remained high (98.7% and 96%, respectively). On SHD, the network still propagated a useful signal (with an accuracy of about 60%), whereas SSC proved more challenging, consistent with its noisier, multiscale temporal structure⁵⁷.

A direct comparison with a purely feedforward SNN shows that once the membrane time constant is the only source of memory, gradients vanish rapidly and performance collapses, whereas the slow-memory variant continues to backpropagate through time. This supports the interpretation that adding a slow, local pathway preserves history without having to enlarge the neuron's intrinsic time constant, and that it can be used as a drop-in component to strengthen existing spiking models that already exploit heterogeneous temporal dynamics.

Hardware implementation of dual memory demonstrates superior throughput and energy efficiency

So far we have shown how the DMP-SNN achieves impressive algorithmic performance alongside excellent parameter number scaling on neuromorphic benchmarks. However, these results do not yet capture all of the advantages of DMP-SNN, which was developed through a co-optimization process to be particularly amenable to hardware implementations. We now introduce a new hardware architecture to realize the full benefits of DMP-SNN.

Our hardware architecture is a digital near-memory-compute architecture that allows the co-optimization of data flow corresponding to the DMPs of the algorithm, instead of the default naive data flow of existing architectures that would simply integrate the heterogeneous data flows of our algorithm. The end-to-end inference micro-architecture for DMP-SNN with a single hidden layer, based on digital near-memory-compute, is depicted in Fig. 4a. At a high level, this architecture supports four parallel computation paths and integrates post-synaptic operations through fine-grained operator fusion and heterogeneous operand stationarity. These design choices break temporal dependencies to balance latency across paths, maximize arithmetic intensity by fusing operators in hardware, and adapt data flow to handle variable sparsity patterns, collectively minimizing off-chip memory traffic. This modular design scales naturally with

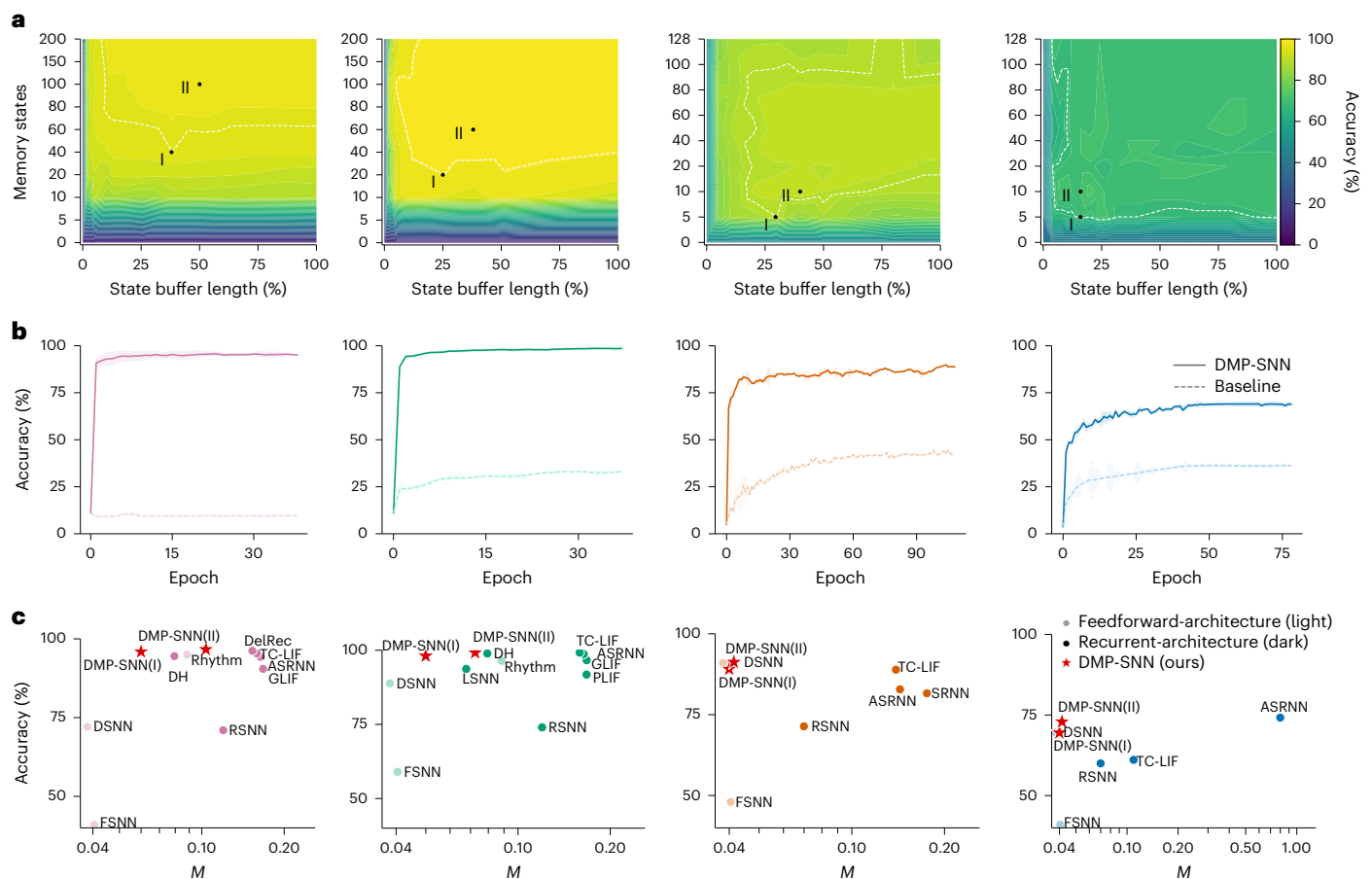


Fig. 2 | Accuracy-efficiency across temporally structured benchmarks.

a, From left to right: test accuracy on PS-MNIST, SMNIST, SHD and SSC for the proposed DMP-SNN. For all datasets, the white dashed curve denotes an iso-accuracy contour, where each point corresponds to a (memory size, state-buffer length) pair achieving the same target accuracy (chosen to be competitive with previous works). The iso-accuracy contours indicate overall trends. The curve highlights two configurations of DMP-SNN: solution I, a single operating point on the curve, and the most parameter-efficient configuration that matches SOTA performance; solution II (accuracy-parameter trade-off), a higher-capacity configuration with low additional parameter cost. **b**, Learning curves for baseline

and the proposed DMP-SNN on PS-MNIST, S-MNIST, SHD and SSC. The central line indicates the mean accuracy across $n = 5$ independent runs, and the shaded area represents the standard error of the mean. Under identical training settings and neuron counts, the DMP-SNN starts from a higher accuracy and converges rapidly, consistently outperforming baseline across datasets. **c**, Comparison of DMP-SNN with recent strong SNN models. DMP-SNN matches or outperforms past work while using a purely feedforward architecture and notably fewer parameters. M , model size. We compared the recent FSNN⁴⁵, DSNN¹⁷, RSNN⁴⁵, LSNN¹⁴, GLIF⁴⁶, PLIF¹⁰, ASRNN¹³, SRNN⁷², DH²⁴, TC-LIF⁷³, Rhythm⁴⁹ and DelRec⁷⁴.

task complexity, enabling flexible trade-offs between temporary storage and parallel throughput. The input stage receives task-agnostic binary spike addresses generated by neuromorphic vision or auditory sensors following the address-event representation protocol⁵⁸, whereas the outputs correspond either to classification or regression results, or to spike streams propagated to downstream layers. We first describe the benchmarking results and then, in the next section, further detail the architectural innovations that made the benchmarking results possible.

We benchmarked DMP-SNN on this co-designed hardware platform against other leading neuromorphic hardware platforms, namely Loihi2⁴², ReckOn⁴¹, ElfCore⁵⁹ and DenRAM²³. Loihi2 is a programmable digital neuromorphic research platform. The benchmark considered in this paper refers specifically to the DSNN implementation proposed in a past work⁴². Our architecture achieves over $4 \times$ higher throughput than a digital delay-based implementation on Loihi2, and over $1.9 \times$ higher throughput than recurrent architectures such as ReckOn (Fig. 4b). These gains arise from compact slow-memory operations and parallelization strategies that eliminate data dependencies. Furthermore, DMP-SNN delivers more than $5 \times$ higher energy efficiency than both the digital delay-based implementation on Loihi2 and

analogue delay-based designs such as DenRAM (Fig. 4c), primarily by eliminating the synaptic-delay buffer overhead (for example, capacitor arrays in analogue implementations). The integration of operator fusion and heterogeneous operand stationarity yields a further $2.5 \times$ improvement in energy efficiency over ElfCore, as confirmed in the ablation (refer to the ‘Experimental set-up’ section in the Methods). Finally, Fig. 4d shows that DMP-SNN incurs only a minor area overhead relative to ElfCore due to intermediate accumulation buffers, yet achieves more than $2 \times$ higher area efficiency than ReckOn, owing to the removal of recurrent weight-matrix storage. Further details of the hardware validation procedure are provided in the Methods. Extended Data Fig. 5 further reports a hardware scaling analysis when doubling the LIF neuron layer.

Overall, the experimental results demonstrate that DMP-SNN attains high computational efficiency on conventional digital hardware, surpassing architectures fabricated on more advanced technology nodes and even designs leveraging emerging memristive devices.

Specific hardware optimizations for DMP-SNN. Our hardware architecture realizes the full efficiencies of the DMP-SNN architecture at the level of advanced data and operator handling optimizations.

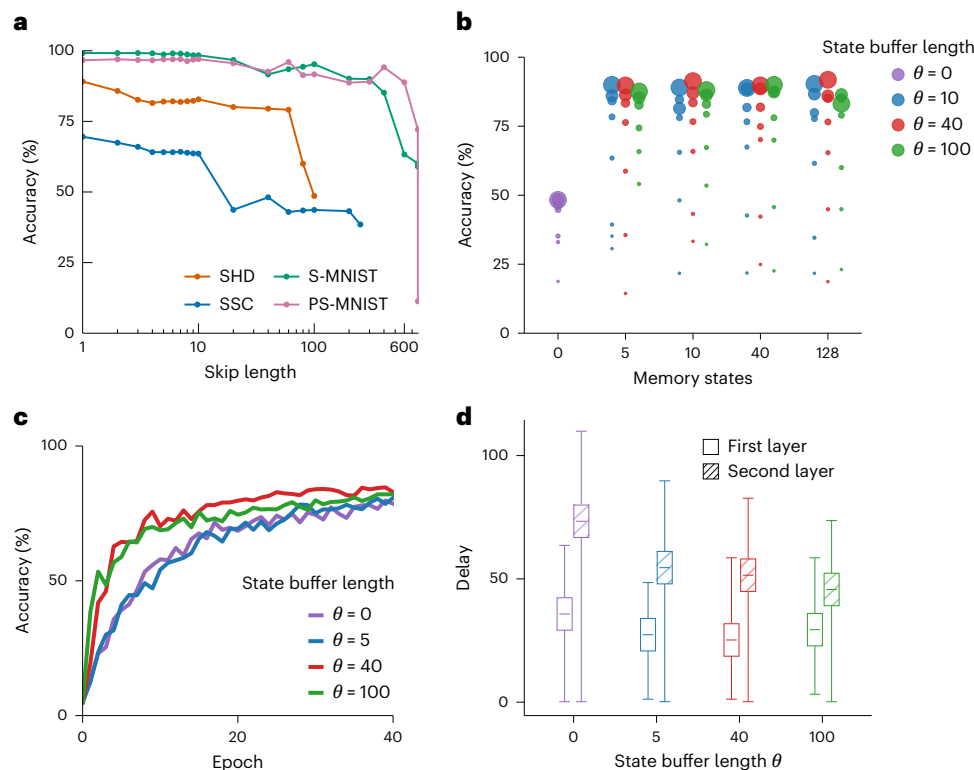


Fig. 3 | Context-dependent temporal demands. **a**, Task-dependent slow memory: increasing the memory-update interval (dilation; log-scale x-axis) leaves SMNIST or PS-MNIST largely unchanged but degrades SHD or SSC, indicating that auditory streams require finer-grained long-term context, whereas long-horizon vision tolerates coarser, less-frequent updates. **b**, Accuracy versus parameter budget and memory dimension saturates once task-relevant timescales are captured, indicating that capacity should be co-tuned rather than maximized. The bubble size denotes the number of feedforward neurons. **c**, Increasing the state buffer length (θ) in the DSNN

accelerates convergence, consistent with long-range context being carried by the slow memory state. **d**, Effect of delay distribution with longer state buffer size. As θ grows, probability mass shifts towards shorter delays, the long-delay tail contracts and overall dispersion decreases, indicating that a longer window partially substitutes for explicit axonal delays. Box plots show the distribution of delays at each state buffer length: the central line represents the mean, the box edges indicate the 25th and 75th percentiles, and the whiskers extend to the minimum and maximum values.

Data-flow-dependency optimization. Although the compact slow memory state introduces little parameter overhead, its long integration path leads to an imbalanced data flow: the fast spike-integration path can start immediately, whereas the memory-integration path cannot. Therefore, accelerating the memory-integration path is necessary to eliminate the throughput bottleneck. As the slow memory update is itself a linear operation, the memory integration with the post-synaptic neuron does not need to wait for the update to complete. Instead, the procedure can be divided into two fully parallel stages, as shown in Fig. 4e. The term on the right, $v x[k]$, still depends on the scalar $x[k]$. However, computing the vector-matrix product $P m[k-1]$ requires approximately the same amount of time as computing $x[k]$ and then forming $v x[k]$. As a result, the computational balance among the critical paths is maintained. In practice, the accumulation of $x[k]$ can be fused with the neuron update u_i , ensuring that the scalar $x[k]$ becomes available concurrently with the binary spike vector from the upstream layer.

Operator-fusion optimization. According to the roofline model⁶⁰, operating in the compute-bound region maximizes arithmetic intensity and hence improves efficiency. In SOTA end-to-end neuromorphic hardware^{41,59}, a finite-state machine typically iterates over all neurons to perform leaky updates before spike integration, incurring at least two memory accesses per neuron. The DMP-SNN model introduces an additional access for the memory-integration path, further complicating efforts to maximize arithmetic intensity. The

hardware implementation fuses leaky updates, spike integration and memory integration into a single process (Fig. 4f). By using temporary register slots for intermediate accumulation, the design reduces memory access to only one per neuron for the entire sequence of computations, thereby enabling operation in the compute-bound regime.

Operand-stationarity optimization. The DMP-SNN algorithm must simultaneously handle spike integration, corresponding to a sparse vector-matrix multiplication (weight accumulation), and memory integration, corresponding to a dense vector-matrix multiplication. Existing SOTA hardware includes Kraken⁶¹, which combines sparse spike-based integration with dense ternary CNN computation, and ElfCore⁵⁹, which integrates sparse spike accumulation with dense eligibility-trace propagation. To optimize both data-flows, DMP-SNN adopts heterogeneous operand stationarity⁶². For spike integration, we use input stationarity to reduce weight-memory access when the input vector is sparse. We adopt output stationarity for memory integration, minimizing neuron-memory access (only one access per neuron) when the input vector is dense. In this scheme, W_p is indexed by the non-zero entries of the spike vector, whereas P and v are indexed by post-synaptic neuron addresses. As illustrated in Fig. 4g, four consecutive neuron states u_i (or more) are preloaded into register slots for intermediate accumulation. To preserve efficient sparse access for W_p , the memory pointers are incremented in a skipped manner based on these four post-synaptic neuron indices.

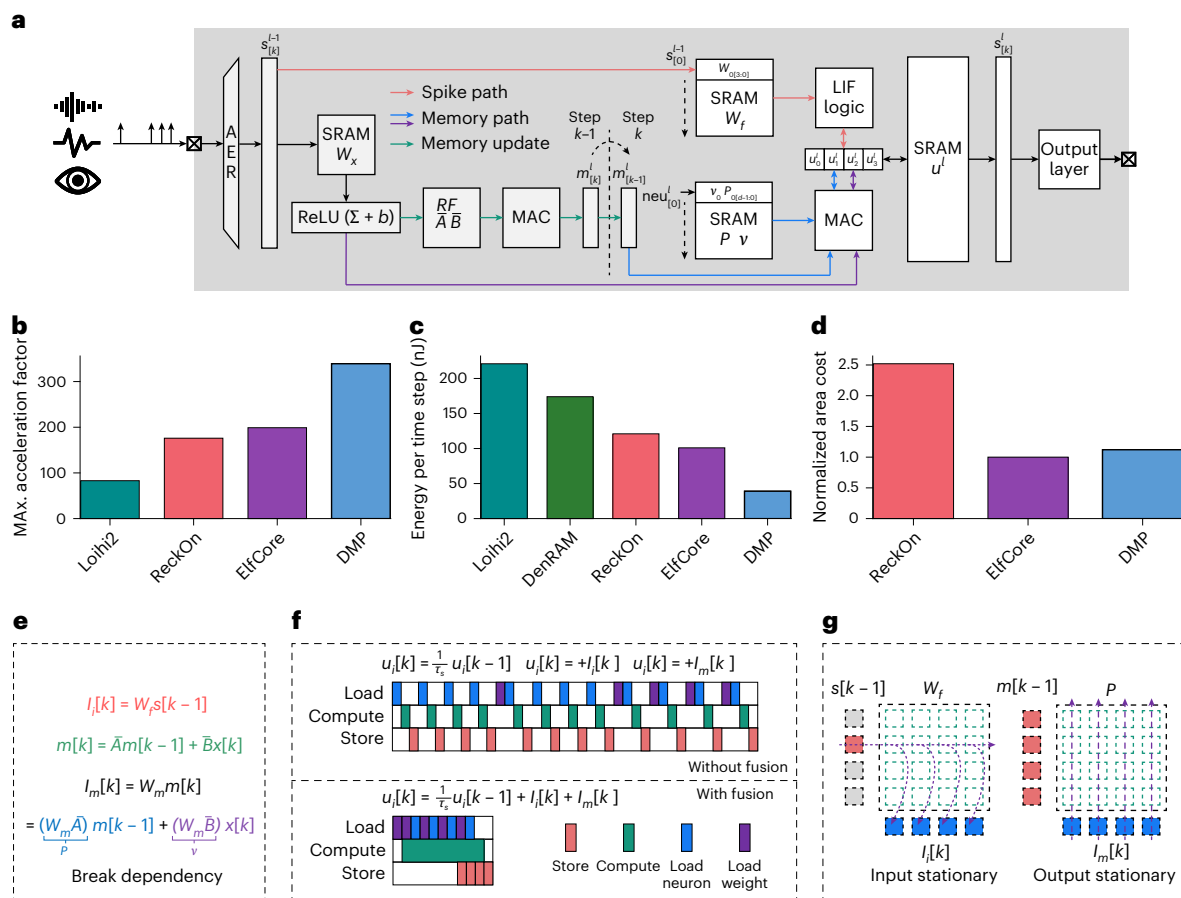


Fig. 4 | Hardware design for the DMP architecture. **a**, Hardware architecture for end-to-end temporal, event-based sensory data inference. Four parallel data paths (spike integration, two memory integration paths and memory update) are shown in distinct colours, enabled by dependency breaking. Dual register slots are allocated for $m[k]$ (memory update) and $m[k-1]$ (memory integration). Four further register slots temporarily store consecutive neuron membrane potentials u_i for fused LIF operations and vector-matrix multiplications before writing back to neuron SRAM. Weight-matrix access patterns differ between spike and memory integration to enhance arithmetic intensity. **b**, DMP-SNN attains the highest throughput per sample by breaking computation dependencies. **c**, DMP-SNN provides up to $5 \times$ higher energy efficiency compared with delay-based hardware designs. **d**, DMP-SNN offers $2 \times$ greater area efficiency than

recurrent SNN architectures, enabled by its reduced parameter-memory footprint. **e**, Dependency breaking between slow memory update and memory integration enables parallel computation. Precomputed weight memories P and v are stored in SRAM. **f**, Comparison of data flow with and without fine-grained operator fusion. The example illustrates updates and integrations of four post-synaptic neuron membrane potentials. Operator fusion allows single-access SRAM reads for neuron states, requiring minimal additional register area while significantly increasing arithmetic intensity. **g**, Heterogeneous operand-stationary schemes are adopted: input-stationary (column-stationary) and output-stationary (row-stationary) access patterns are used for sparse spike integration and dense vector-matrix multiplications, respectively, further improving arithmetic intensity.

Discussion

Our work addresses a persistent tension: spiking models require access to long behavioural timescales, yet the mechanisms that typically provide this context—such as dense recurrence or long learnable delays—are precisely those that burden hardware⁴¹. These mechanisms distribute state across many connections, increase memory traffic and demand deep buffers, causing temporal capability and implementation efficiency to diverge²⁶. Here we show that this tension is not inherent and can be resolved through hardware-algorithm co-design. If temporal context is made explicit, compact and local to each layer, then long-range processing can coexist with hardware efficiency rather than compete with it².

On the algorithm level, we draw on a widespread cortical motif in which fast somatic spiking is shaped by slower, local dendritic integration^{39,40}. We abstract the biologically observed fast-slow separation into an engineering design principle: a second state that is proximal to the layer but not per-synapse. Each layer maintains a low-dimensional slow state that summarizes recent activity and feeds back as an additional current. Being layer-shared, the state's cost scales linearly rather than quadratically with network width;

because its dynamics are well conditioned^{63,64}, it offers a more stable gradient pathway than the decaying membrane. Under strict last-time step supervision, this configuration sustained useful gradients over hundreds of steps on S-MNIST or PS-MNIST, whereas a purely feed-forward SNN did not, indicating that long-term credit assignment is carried by the explicit slow state rather than by inflating neuronal time constants.

The hardware mirrors this separation. Instead of forcing sparse spike accumulation and dense linear updates through a single uniform core, we implement two coupled paths: one for spike-driven computation and one for the shared state. Keeping the state on-chip, decoupling update from consume, and adopting distinct data-flow policies for sparse and dense kernels increase arithmetic intensity without extra off-chip traffic. Post-layout results in 22 nm confirm that this mapping holds in silicon: performance matches or exceeds recurrent and delay-based baselines at lower energy and higher throughput, with only a modest area overhead from the additional state. In practice, these hardware gains arise directly from the same design principle used at the algorithmic level: isolate slow information, keep it compact, and move it through a dedicated path.

The DMP-SNN hardware targets a different point in the design space than large-scale neuromorphic platforms such as Loihi2 or SpiNNaker 2. It is primarily designed for edge computing. For large-scale neuromorphic systems, DMP-SNN can serve as an independent core or be integrated into platforms such as Loihi2 or SpiNNaker. Deploying it on these systems would enable broader evaluation of hardware efficiency and platform compatibility. More importantly, DMP-SNN addresses a key gap in neuromorphic design: the efficient integration of sparse and dense data paths. By incorporating dense data processing into SNNs through heterogeneous hardware integration, it improves model accuracy while maintaining high hardware efficiency.

Finally, our results suggest a clear algorithm–hardware design principle for long-timescale spiking: make the slow state explicit, shared and low-rank; keep the fast path spike-driven; and execute the two in parallel on hardware. Under this formulation, long temporal horizons no longer compete with efficiency, and the biological fast–slow motif provides a high-level design pattern rather than a loose analogy. This establishes a scalable template for neuromorphic systems in which temporal competence and deployment practicality reinforce each other, rather than exist in trade-off.

Methods

Spiking neurons

We model spiking neurons as LIF units. For neuron i in layer l at discrete time step k , the membrane potential is $u_i^l[k]$. Its dynamics are governed by the membrane decay β and the input current $I_i^l[k]$, with the discrete-time update

$$u_i^l[k] = \beta u_i^l[k-1] + I_i^l[k], \quad (1)$$

The input current is

$$I_i^l[k] = W_{ij} s^{l-1}[k] \quad (2)$$

where W_{ij} denotes feedforward synaptic weights and $s^{l-1}[k]$ the presynaptic spike vector from layer $l-1$.

A spike is emitted when the membrane exceeds threshold:

$$s_i^l[k] = \Theta(u_i^l[k] - \theta_u), \quad (3)$$

where $\Theta(\cdot)$ is the Heaviside step function (1 if its argument is positive and 0 otherwise), and θ_u is the membrane threshold.

Building on this vanilla LIF neuron, we consider two variants that extend temporal capacity: ‘LIF with axonal delays’ models transmission delays as a temporal shift of presynaptic spikes, that is, $s^{l-1}[k] \rightarrow s^{l-1}[k-\bar{d}]$; ‘LIF with recurrent connections’ augments the update by adding an additional recurrent term within the layer.

Slow memory pathway

We implement the slow pathway using a state-space model that captures long-term dependencies through well-conditioned linear dynamics. We treat the slow pathway as a generic linear memory module (for example, linear RNN or SSM-style). In our experiments, we instantiate it with an Legendre memory unit (LMU)-like state-space realization for its explicit parameterization and stable long-horizon dynamics. This design builds on the LMU framework^{63,65}, which formulates a continuous-time memory as a linear time-invariant system. The LMU combines a single-input delay network with a nonlinear dynamical system, efficiently encoding recent input history through orthogonalized basis functions derived from Legendre polynomials. The delay network projects the input onto a set of orthogonal basis functions derived from Legendre polynomials, providing an efficient representation of temporal context for subsequent nonlinear computation. The delay network orthogonalizes the input signal over a sliding window of length θ (we denote it as state buffer length), whereas the nonlinear system

leverages this orthogonalized memory to compute various functions over time. Mathematically, given an input scalar function $x(t)$, the state update can be described as follows:

$$m'(t) = Am(t) + Bx(t) \quad (4)$$

where $m(t) \in \mathbb{R}^d$ denotes the memory state vector with dimension d , and A and B are state-space matrices. Following the use of Padé approximation^{64,66}, the state-space matrices can be expressed as follows:

$$A = [a]_{ij} \in \mathbb{R}^{d \times d}, a_{ij} = (2i+1) \begin{cases} -1 & \text{if } i < j \\ (-1)^{i-j+1} & \text{if } i \geq j \end{cases} \quad (5)$$

$$B = [b]_i \in \mathbb{R}^{d \times 1}, b_i = (2i+1)(-1)^i, i, j \in [0, d-1]$$

This continuous-time system can be converted to discrete-time k with a time resolution Δt :

$$m^l[k] = \bar{A}m^l[k-1] + \bar{B}x^l[k] \quad (6)$$

Here, $\bar{A}$ and $\bar{B}$ are the discretized versions of A and B , obtained using the zero-order hold method. Specifically, exact discretization yields

$$\bar{A} = e^{A\Delta t} \text{ and } \bar{B} = A^{-1}(e^{A\Delta t} - I)B,$$

and these matrices are usually frozen during training.

DMP-SNN

Each layer maintains a low-dimensional slow state $m^l[k] \in \mathbb{R}^d$ that summarises recent activity and feeds back as an additional current. As suggested by Chilkuri et al.⁶⁷, we compress presynaptic spikes into a scalar drive:

$$x^l[k] = f_x(W_x s^{l-1}[k] + b) \quad (7)$$

Where f_x is the mapping function. The shared memory is updated by

$$m^l[k] = \bar{A}m^l[k-1] + \bar{B}x^l[k]. \quad (8)$$

The membrane update is

$$u^l[k] = \beta u^l[k-1] + I^l[k] + I_m^l[k] \quad (9)$$

The input current I and memory input I_m are given by:

$$I^l[k] = W_{ij} s^{l-1}[k], \quad (10)$$

$$I_m^l[k] = W_m m^l[k] \quad (11)$$

where W_m represents the weights from memory. This formulation couples a fast spiking pathway with a slow memory that evolves stably over long timescales. The full detailed equation is listed in Extended Data Table 3. We used the network parameters given in Extended Data Table 4 for all experiments, unless otherwise specified. Our method is trained via the SpikingJelly and SLAYER frameworks^{68,69} and used the mean membrane potential of the last layer as the output.

Analysis of the gradient of DMP-SNN

We analyse temporal credit assignment under last-time step supervision. For a hidden layer l , let $u^l[k] \in \mathbb{R}^N$ and $m^l[k] \in \mathbb{R}^d$ denote the membrane potentials and slow memory at discrete time k , respectively. Ignoring input-driven terms (which act as biases and do not affect state-state Jacobians), the dynamics within the layer can be written as

$$u^l[k+1] = \beta u^l[k] + W_m m^l[k+1], \quad (12)$$

$$m^l[k+1] = \bar{A} m^l[k], \quad (13)$$

with leak factor $\beta \in (0, 1)$, memory readout matrix $W_m \in \mathbb{R}^{N \times d}$, and state-transition matrix $\bar{A} \in \mathbb{R}^{d \times d}$ arising from the state space construction.

It is convenient to group membrane and memory into a joint state

$$z^l[k] = \begin{bmatrix} u^l[k] \\ m^l[k] \end{bmatrix} \in \mathbb{R}^{N+d}.$$

Equations (12) and (13) define a linear time-invariant update

$$z^l[k+1] = F z^l[k], \quad F = \begin{bmatrix} \beta I_N & W_m \bar{A} \\ 0 & \bar{A} \end{bmatrix},$$

where I_N denotes the $N \times N$ identity matrix. Over a temporal horizon from k to T , the same-layer Jacobian is therefore

$$\frac{\partial z^l[T]}{\partial z^l[k]} = F^{T-k}. \quad (14)$$

As F is block upper-triangular, its eigenvalues are

$$\text{spec}(F) = \{\beta \text{ (multiplicity } N)\} \cup \text{spec}(\bar{A}),$$

where $\text{spec}(\bar{A}) = \{\lambda_i(\bar{A})\}$ denotes the spectrum of $\bar{A}$ and $\lambda_i(\bar{A})$ is the i th eigenvalue of $\bar{A}$. The eigenvalue β arises from the membrane block βI_N and the remaining eigenvalues arise from the memory block $\bar{A}$. For a purely FSNN, the state reduces to the membrane only, $z^l[k] = u^l[k]$, and $F = \beta I_N$, so that

$$\frac{\partial u^l[T]}{\partial u^l[k]} = \beta^{T-k} I_N,$$

and all gradient components decay at rate $|\beta|^{T-k}$. In the DMP-SNN, the modes associated with the membrane block still decay at this fast rate, but the memory block $\bar{A}$ contributes further modes whose decay is governed by $|\lambda_i(\bar{A})|^{T-k}$. As the slow memory feeds back into the membrane via the term $W_m m^l[k]$ in equation (12), these memory modes are continuously injected into the membrane dynamics and appear as slow components in $u^l[T]$. By construction, the spectral radius $\rho(\bar{A})$ is close to one⁶³, so these memory-aligned modes carry gradients over substantially longer horizons.

Intuitively, the DMP-SNN therefore provides two distinct temporal scales for credit assignment: a fast mode, inherited from the leaky membrane, and a slow mode, inherited from the well-conditioned memory dynamics. Empirically, this leads to a broad temporal tail in the gradient profile over k for the DMP-SNN, whereas the FSNN has a sharp peak near $k = T$ and rapid decay towards earlier time steps (Extended Data Fig. 4).

Dilated slow memory

Dilated slow memory updates the memory-driven input only every fixed number of time steps, thereby skipping intermediate computations on hardware. The memory-driven input to neuron n at time step k is

$$I_m^l[k] = W_m m^{(l)}[k_d], \quad k_d = \lfloor k/d_s \rfloor d_s \quad (15)$$

where d_s is the skip length (dilation factor), $m^l[k_d]$ denotes the memory vector subsampled every d_s steps (that is, $[m[0], m[d_s], m[2d_s], \dots, m[\lfloor k/d_s \rfloor d_s]]^\top$). Between updates (when $k \bmod d_s \neq 0$), the membrane potential evolves without the memory injection.

Hardware baselines

State-of-the-art hardware implementations of efficient long-term memory for temporal signal processing include:

Loihi2⁴²: an end-to-end framework that enables event-based SNN training with synaptic delays on GPUs and deployment on Intel's Loihi2 neuromorphic chip. It represents the SOTA in digital hardware implementations featuring synaptic delays. Loihi2 is not specifically optimized for synaptic delay implementation; however, to the best of our knowledge, it provides the only publicly available SOTA results for a digital implementation of SNN delays.

DenRAM²³: an analogue feedforward SNN with dendritic compartments integrated with resistive random access memory in 130 nm technology, supporting both synaptic delays and weights. It represents the SOTA analogue hardware approach using memristor-based delayed synapses.

ReckOn⁴¹: a spiking RNN processor that supports task-agnostic online learning over multi-second timescales. It represents the SOTA in digital spiking RNN implementations.

ElfCore⁵⁹: a SNN processor equipped with a local self-supervised learning engine for multilayer temporal learning without labelled inputs. It represents the SOTA in digital hardware integrating both sparse spike and dense trace data paths. For the ablation study, we implemented a modified ElfCore-based architecture without operator fusion or heterogeneous operand stationarity⁵⁹.

Experimental set-up

Implementation and evaluation methodology. The proposed DMP-SNN architecture and the ElfCore-based design were implemented in an advanced 22FDX process node and evaluated through post-layout simulations using QuestaSim and Innovus.

Hardware performance metrics were obtained by averaging inference results over the complete SHD test set. All networks were trained offline, and the reported accuracies correspond to hardware-based inference measurements.

Reference baselines results. We compare our design against SOTA neuromorphic platforms, including Loihi2, DenRAM and ReckOn. The reported metrics for these platforms were extracted from their respective publications and normalized to account for differences in process technology and supply voltage.

- **Loihi2**: achieves 88% hardware-measured accuracy and represents the SOTA digital delay-based implementation.
- **DenRAM**: achieves approximately 87% accuracy and represents the SOTA memristor-based delay implementation. This value is based on hardware-aware simulations rather than full end-to-end silicon measurements.
- **ReckOn**: achieves 86.2% hardware-measured accuracy and represents a SOTA recurrence-based implementation.
- **ElfCore**: achieves 90.3% accuracy on the full SHD test set and serves as the hardware baseline for ablation analysis.

The proposed DMP-SNN also achieves 90.3% accuracy. Both ElfCore and DMP-SNN deploy the identical trained network model. The key difference lies in the architectural enhancements—such as operator fusion and the heterogeneous operator-stationary strategy, which are applied exclusively in DMP-SNN. Therefore, the comparison between ElfCore and DMP-SNN isolates and quantifies the hardware-level benefits introduced by the proposed optimizations.

Comparison metrics. The evaluation considers the following hardware-level metrics:

- **Maximum acceleration factor**: The ratio between the core processing speed per time step and real-time execution (1 ms per SHD time step).

- **Energy per time step:** The average energy consumed to process a single inference time step.
- **Area cost:** The post-layout, tape-out-ready silicon area of the computing core, reflecting hardware area efficiency under matched (and improved) functionality.

Rationale for selecting ReckOn as the area baseline. ReckOn is chosen as the area comparison baseline for three primary reasons:

1. It is the only past work that reports the complete computing-core area in silicon.
2. Its architecture closely resembles ours, adopting a single-core design targeted at extreme edge computing scenarios such as sensor-level processing.
3. It fully uses on-chip resources for the same SHD task, ensuring a fair and meaningful area comparison.

Data availability

The spiking data used in this study are publicly available and open source. The dataset for SHD and SSC belong to Spiking Heidelberg Datasets, which can be accessed via <https://zenkelab.org/datasets/>. S-MNIST and PS-MNIST are derived from the original MNIST dataset, which can be accessed via <http://yann.lecun.com/exdb/mnist/>.

Code availability

The source code for the algorithm and hardware implementation are available at https://github.com/sunpengfei122/Dual_memory_pathways (ref. 70) and Zenodo via <https://doi.org/10.5281/zenodo.19839869> (ref. 71).

References

1. Maass, W. Networks of spiking neurons: the third generation of neural network models. *Neur. Netw.* **10**, 1659–1671 (1997).
2. Roy, K., Jaiswal, A. & Panda, P. Towards spike-based machine intelligence with neuromorphic computing. *Nature* **575**, 607–617 (2019).
3. Gerstner, W. & Kistler, W.M. *Spiking Neuron Models: Single Neurons, Populations, Plasticity* 157 (Cambridge Univ. Press, 2002).
4. Li, G. et al. Brain-inspired computing: a systematic survey and future trends. *Proc. IEEE* **112**, 544–584 (2024).
5. Yu, Z., Sun, P. & Goodman, D. F. M. Beyond rate coding: surrogate gradients enable spike timing learning in spiking neural networks. *Neurom. Comput. Eng.* **6**, 014016 (2026).
6. Mostafa, H. Supervised learning based on temporal coding in spiking neural networks. *IEEE Trans. Neur. Netw. Learn. Syst.* **29**, 3227–3235 (2017).
7. Comşa, I.-M. et al. Temporal coding in spiking neural networks with alpha synaptic function: learning with backpropagation. *IEEE Trans. Neur. Netw. Learn. Syst.* **33**, 5939–5952 (2021).
8. Sun, P., Wu, J., Zhang, M., Devos, P. & Botteldooren, D. Delay learning based on temporal coding in spiking neural networks. *Neur. Netw.* **180**, 106678 (2024).
9. Gast, R., Solla, S. A. & Kennedy, A. Neural heterogeneity controls computations in spiking neural networks. *Proc. Natl Acad. Sci. USA* **121**, e2311885121 (2024).
10. Fang, W. et al. Incorporating learnable membrane time constant to enhance learning of spiking neural networks. In *Proc. IEEE/CVF International Conference on Computer Vision* 2661–2671 (IEEE/CVF, 2021).
11. Perez-Nieves, N., Leung, V. C. H., Dragotti, P. L. & Goodman, D. F. M. Neural heterogeneity promotes robust learning. *Nat. Commun.* **12**, 5791 (2021).
12. Shaban, A., Bezugam, S. S. & Suri, M. An adaptive threshold neuron for recurrent spiking neural networks with nanodevice hardware implementation. *Nat. Commun.* **12**, 4234 (2021).
13. Yin, B., Corradi, F. & Bohtë, S. M. Accurate and efficient time-domain classification with adaptive spiking recurrent neural networks. *Nat. Mach. Intell.* **3**, 905–913 (2021).
14. Bellec, G., Salaj, D., Subramoney, A., Legenstein, R. & Maass, W. Long short-term memory and learning-to-learn in networks of spiking neurons. In *32nd Conference on Neural Information Processing Systems (NeurIPS)*, 2018).
15. Sreenivasan, K. K. & D’Esposito, M. The what, where and how of delay activity. *Nat. Rev. Neurosci.* **20**, 466–481 (2019).
16. Izhikevich, E. M. Polychronization: computation with spikes. *Neur. Comput.* **18**, 245–282 (2006).
17. Sun, P., Chua, Y., Devos, P. & Botteldooren, D. Learnable axonal delay in spiking neural networks improves spoken word recognition. *Front. Neurosci.* **17**, 1275944 (2023).
18. Wang, X., Lin, X. & Dang, X. A delay learning algorithm based on spike train kernels for spiking neurons. *Front. Neurosci.* **13**, 252 (2019).
19. Yu, Q. et al. Improving multispike learning with plastic synaptic delays. *IEEE Trans. Neur. Netw. Learn. Syst.* **34**, 10254–10265 (2022).
20. Grappolini, E. & Subramoney, A. Beyond weights: deep learning in spiking neural networks with pure synaptic-delay training. In *Proc. 2023 International Conference on Neuromorphic Systems* 1–4 (ICONS, 2023).
21. Hammouamri, I., Hassani, I.K. & Masquelier, T. Learning delays in spiking neural networks using dilated convolutions with learnable spacings. In *International Conference on Learning Representations (ICLR)*, 2024).
22. Patiño-Saucedo, A. et al. Empirical study on the efficiency of spiking neural networks with axonal delays, and algorithm–hardware benchmarking. In *2023 IEEE International Symposium on Circuits and Systems (ISCAS)* 1–5 (IEEE, 2023).
23. D’agostino, S. et al. DenRAM: neuromorphic dendritic architecture with RRAM for efficient temporal processing with delays. *Nat. Commun.* **15**, 3446 (2024).
24. Zheng, H. et al. Temporal dendritic heterogeneity incorporated with spiking neural networks for learning multi-timescale dynamics. *Nat. Commun.* **15**, 277 (2024).
25. Mészáros, B., Knight, J. C. & Nowotny, T. Efficient event-based delay learning in spiking neural networks. *Nat. Commun.* **16**, 10422 (2025).
26. Patiño-Saucedo, A. et al. Hardware-aware training of models with synaptic delays for digital event-driven neuromorphic processors. Preprint at <https://arxiv.org/abs/2404.10597> (2024).
27. Karilanova, S., Dey, S. & Özçelikkale, A. Delays in spiking neural networks: a state space model approach. Preprint at <https://arxiv.org/abs/2512.01906> (2025).
28. Maass, W., Natschläger, T. & Markram, H. Real-time computing without stable states: A new framework for neural computation based on perturbations. *Neur. Comput.* **14**, 2531–2560 (2002).
29. Biswas, A., Medhe, S.A., Singhal, R. and Ganguly, U. Temporal and spatial reservoir ensembling techniques for liquid state machines. In *2024 International Conference on Neuromorphic Systems (ICONS)* 78–81 (IEEE, 2024).
30. Deckers, L., Tsang, I. J., Van Leekwijck, W. & Latré, S. Extended liquid state machines for speech recognition. *Front. Neurosci.* **16**, 1023470 (2022).
31. Thakur, C. S. et al. Large-scale neuromorphic spiking array processors: A quest to mimic the brain. *Front. Neurosci.* **12**, 891 (2018).
32. Lenz, G., Ieng, S.-H. & Benosman, R. Event-based face detection and tracking using the dynamics of eye blinks. *Front. Neurosci.* **14**, 587 (2020).

33. Payvand, M., Fouda, M. E., Kurdahi, F., Eltawil, A. M. & Neftci, E. O. On-chip error-triggered learning of multi-layer memristive spiking neural networks. *IEEE J. Emerging Sel. Top. Circ. Syst.* **10**, 522–535 (2020).
34. Wu, Y. et al. Brain-inspired global-local learning incorporated with neuromorphic computing. *Nat. Commun.* **13**, 65 (2022).
35. Dampfhofer, M., Mesquida, T., Valentian, A. & Anghel, L. Backpropagation-based learning techniques for deep spiking neural networks: a survey. *IEEE Trans. Neur. Netw. Learn. Syst.* **35**, 11906–11921 (2023).
36. Zheng, Z. et al. Modeling macroscopic brain dynamics with brain-inspired computing architecture. *Nat. Commun.* **16**, 9424 (2025).
37. London, M. & Häusser, M. Dendritic computation. *Annu. Rev. Neurosci.* **28**, 503–532 (2005).
38. Murray, J. D. et al. A hierarchy of intrinsic timescales across primate cortex. *Nat. Neurosci.* **17**, 1661–1663 (2014).
39. Sartzetaki, C. et al. The human brain as a dynamic mixture of expert models in video understanding. Preprint at *bioRxiv* <https://doi.org/10.1101/2025.10.27.684803> (2025).
40. Cook, J., Akarca, D., Costa, R. & Achterberg, J. Brain-like processing pathways form in models with heterogeneous experts. In *The 39th Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2025).
41. Frenkel, C. & Indiveri, G. ReckON: a 28 nm sub-mm² task-agnostic spiking recurrent neural network processor enabling on-chip learning over second-long timescales. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)* Vol. 65 (IEEE, 2022).
42. Mészáros, B., Knight, J.C., Timcheck, J. & Nowotny, T. A complete pipeline for deploying snns with synaptic delays on loihi 2. Preprint at <https://arxiv.org/abs/2510.13757> (2025).
43. Sun, P., Achterberg, J., Su, Z., Goodman, D.F. & Akarca, D. Exploiting heterogeneous delays for efficient computation in low-bit neural networks. Preprint at <https://arxiv.org/abs/2510.27434> (2025).
44. LeCun, Y., Bottou, L., éon, Bengio, Y. & Haffner, P. Gradient-based learning applied to document recognition. *Proc. IEEE* **86**, 2278–2324 (2002).
45. Cramer, B., Stradmann, Y., Schemmel, J. & Zenke, F. The Heidelberg spiking data sets for the systematic evaluation of spiking neural networks. *IEEE Trans. Neur. Netw. Learn. Syst.* **33**, 2744–2757 (2020).
46. Yao, X., Li, F., Mo, Z. & Cheng, J. GLIF: a unified gated leaky integrate-and-fire neuron for spiking neural networks. In *36th Conference on Neural Information Processing Systems (NeurIPS 2022)* <https://openreview.net/pdf?id=UmFSx2c4ubT> (2022).
47. Sun, P., Wu, J., Zhang, M., Devos, P. & Botteldooren, D. Delayed memory unit: modeling temporal dependency through delay gate. *IEEE Trans. Neur. Netw. Learn. Syst.* **36**, 10808–10818 (2024).
48. Fan, L. et al. A multisynaptic spiking neuron for simultaneously encoding spatiotemporal dynamics. *Nat. Commun.* **16**, 7155 (2025).
49. Yan, Y. et al. Efficient and robust temporal processing with neural oscillations modulated spiking neural networks. *Nat. Commun.* **16**, 8651 (2025).
50. Du, Y., Liu, X. & Chua, Y. Spiking structured state space model for monaural speech enhancement. In *ICASSP 2024–2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* 766–770 (IEEE, 2024).
51. Liu, Z., Datta, G., Li, A. & Beerel, P. A. LMUFORMER: low complexity yet powerful spiking model with legendre memory units. In *International Conference on Learning Representations* <https://openreview.net/pdf?id=oEF7qExD9F> (ICLR, 2024).
52. Stan, M.-I. & Rhodes, O. Learning long sequences in spiking neural networks. *Sci. Rep.* **14**, 21957 (2024).
53. Sun, P., Jiang, W., Devos, P. & Botteldooren, D. Parallel delayed memory units for enhanced temporal modeling in biomedical and bioacoustic signal analysis. *IEEE Trans. Audio Speech Language Proces.* **33**, 4797–4807 (2025).
54. Hammouamri, I., Khalfaoui Hassani, I. & Masquelier, T. Learning delays in spiking neural networks using dilated convolutions with learnable spacings. In *International Conference on Learning Representations* <https://openreview.net/pdf?id=4r2ybzJnmN> (ICLR, 2024).
55. Sun, P., Wu, J., Devos, P. & Botteldooren, D. Towards parameter-free attentional spiking neural networks. *Neur. Netw.* **185**, 107154 (2025).
56. Grimaldi, A. & Perrinet, L. U. Learning heterogeneous delays in a layer of spiking neurons for fast motion detection. *Biol. Cyb.* **117**, 373–387 (2023).
57. Alkilany, A. & Goodman, D.F. Neuromodulation enhances dynamic sensory processing in spiking neural network models. Preprint at *bioRxiv* <https://doi.org/10.1101/2025.07.25.666748> (2025).
58. Su, Z. et al. An ultra-low cost and multicast-enabled asynchronous noc for neuromorphic edge computing. *IEEE J. Emerg. Sel. Top. Circ. Syst.* **14**, 409–424 (2024).
59. Su, Z. & Indiveri, G. Elfcore: a 28nm neural processor enabling dynamic structured sparse training and online self-supervised learning with activity-dependent weight update. In *2025 IEEE European Solid-State Electronics Research Conference (ESSERC)* 13–16 (IEEE, 2025).
60. Verhelst, M., Benini, L. & Verma, N. How to keep pushing ml accelerator performance? know your rooflines!. *IEEE J. Sol. State Circ.* **60**, 1888–1905 (2025).
61. Potocnik, V. et al. Circuits and systems for embodied AI: Exploring uJ multi-modal perception for nano-UAVs on the Kraken Shield. In *2024 IEEE European Solid-State Electronics Research Conference (ESSERC)* 1–4 (IEEE, 2024).
62. Kwon, H. et al. Understanding reuse, performance, and hardware cost of DNN dataflow: a data-centric approach. In *Proc. 52nd Annual IEEE/ACM International Symposium on Microarchitecture* 754–768 (IEEE, 2019).
63. Voelker, A., Kajić, I. & Elias Smith, C. Legendre memory units: continuous-time representation in recurrent neural networks. In *33rd Conference on Neural Information Processing Systems* 15570–15579 (NeurIPS, 2019).
64. Gu, A., Dao, T., Ermon, S., Rudra, A. & Ré, C. HiPPO: recurrent memory with optimal polynomial projections. In *34th Conference on Neural Information Processing Systems* 1474–1487 (NeurIPS, 2020).
65. Gu, A., Goel, K. & Ré, C. Efficiently modeling long sequences with structured state spaces. Preprint at <https://arxiv.org/abs/2111.00396> (2021).
66. Padé, H. Sur la représentation approchée d’une fonction par des fractions rationnelles. *Ann. Sci. l’Ecole Norm. Sup.* **9**, 3–93 (1892).
67. Chilkuri, N.R. & Elias Smith, C. Parallelizing legendre memory unit training. In *International Conference on Machine Learning* 1898–1907 (PMLR, 2021).
68. Fang, W. et al. Spikingjelly: an open-source machine learning infrastructure platform for spike-based intelligence. *Sci. Adv.* **9**, eadi1480 (2023).
69. Shrestha, S.B. & Orchard, G. SLAYER: spike layer error reassignment in time. In *32nd Conference on Neural Information Processing Systems* https://proceedings.neurips.cc/paper_files/paper/2018/file/82f2b308c3b01637c607ce05f52a2fed-Paper.pdf (NeurIPS, 2018).
70. Sun, P. Dual_memory_pathways. *GitHub* https://github.com/sunpengfei1122/Dual_memory_pathways (2026).

71. Sun, P. et al. Dual_memory_pathways: algorithm–hardware co-design of Nneuromorphic networks with dual memory pathway. *Zenodo* <https://doi.org/10.5281/zenodo.19839869> (2026).
72. Pagkalos, M., Chavlis, S. & Poirazi, P. Introducing the dendrify framework for incorporating dendrites to spiking neural networks. *Nat. Commun.* **14**, 131 (2023).
73. Zhang, S. et al. TC-LIF: a two-compartment spiking neuron model for long-term sequential modelling. *Proc. AAAI Conf. Artif. Intell.* **38**, 16838–16847 (2024).
74. Queant, A., Rançon, U., Cottureau, B.R. & Masquelier, T. DelRec: learning delays in recurrent spiking neural networks. Preprint at <https://arxiv.org/abs/2509.24852> (2025).

Acknowledgements

This project was funded by the Advanced Research + Invention Agency (ARIA), partially supported by the HORIZON EUROPE EIC Pathfinder Grant ELEGANCE (grant no. 101161114), and received funding from Swiss National Science Foundation (SNSF 200021E_222393). D.A. is funded by an Imperial College Research Fellowship, Schmidt Sciences Fellowship and Templeton World Charity Foundation (funder DOI 501100011730) under grant no. TWCF-2022-30510. J.A. is funded by a Career Development Research Fellowship of St John's College, Oxford. For the purpose of open access, the authors have applied a Creative Commons Attribution (CC BY) license to the text, figures and code relating to this paper.

Author contributions

P.S. and Z.S. contributed equally, designing algorithms, performing experiments and analysing results. J.A., G.I., D.F.M.G. and D.A. guided methodology, coordinated the study, and all authors contributed to experimental design, analysis, writing and manuscript review.

Competing interests

The authors declare no competing interests.

Additional information

Extended data is available for this paper at <https://doi.org/10.1038/s42256-026-01255-3>.

Supplementary information The online version contains supplementary material available at <https://doi.org/10.1038/s42256-026-01255-3>.

Correspondence and requests for materials should be addressed to Pengfei Sun, Zhe Su or Danyal Akarca.

Peer review information *Nature Machine Intelligence* thanks James Aimone, and the other, anonymous, reviewer(s) for their contribution to the peer review of this work.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

© The Author(s) 2026

¹Department of Electrical and Electronic Engineering, Imperial College London, London, UK. ²Institute of Neuroinformatics, University of Zurich and ETH Zurich, Zürich, Switzerland. ³Centre for Neural Circuits and Behaviour, University of Oxford, Oxford, UK. ⁴Imperial-X, Imperial College London, London, UK. ⁵MRC Cognition and Brain Sciences Unit, University of Cambridge, Cambridge, UK. ⁶These authors contributed equally: Pengfei Sun, Zhe Su. ✉e-mail: p.sun@imperial.ac.uk; zhesu@ini.ethz.ch; d.akarca@imperial.ac.uk

Extended Data Table 1 | DVS Gesture results. Test accuracy and parameter count for the baseline feedforward spiking network (FSNN) and the proposed DMP variant

Dataset	FSNN	DMP
DVS Gesture	87.12% / 1.07M	91.32% / 1.09M

We evaluated DMP on the DVS Gesture dataset, which comprises event-based spatiotemporal streams and is commonly modelled using spiking convolutional pipelines. As a baseline, we used two spiking convolutional layers followed by two spiking fully connected layers, with pooling applied after each convolution and after the input stage. DMP was inserted into the first spiking fully connected layer, where temporal integration over the extracted features becomes important. All results were obtained with a simulation length of 1000 time steps. As shown in Table-1, incorporating DMP increases accuracy from 87.12% to 91.32% with a marginal increase in parameters (1.07M to 1.09M).

Extended Data Table 2 | Performance comparison across state-space variants

Dataset	SSM Accuracy	SSM Parameters	Spiking SSM Accuracy	Spiking SSM Parameters	DMP ($W_f=0$) Accuracy	DMP ($W_f=0$) Parameters	DMP Accuracy	DMP Parameters
SHD	90.06%	73K	85.09%	73K	5.00%	37K	91.23%	40K
SSC	67.44%	75K	69.13%	75K	2.90%	39K	72.90%	42K
S-MNIST	99.03%	202K	99.02%	202K	10.00%	42K	99.20%	73K
PS-MNIST	97.21%	202K	95.49%	202K	10.00%	42K	96.65%	102K

Test accuracy (% , left of each entry) and total parameter count (right; in K) for three architectures: (i) a non-spiking state-space model (SSM), (ii) a spiking state-space model (Spiking SSM), (iii) the proposed dual memory pathway spiking network (DMP-SNN), and DMP removing the feedforward layer($W_f=0$). Results are reported on SHD and SSC (event-based auditory classification) and on S-MNIST and PS-MNIST (long-horizon sequential vision).

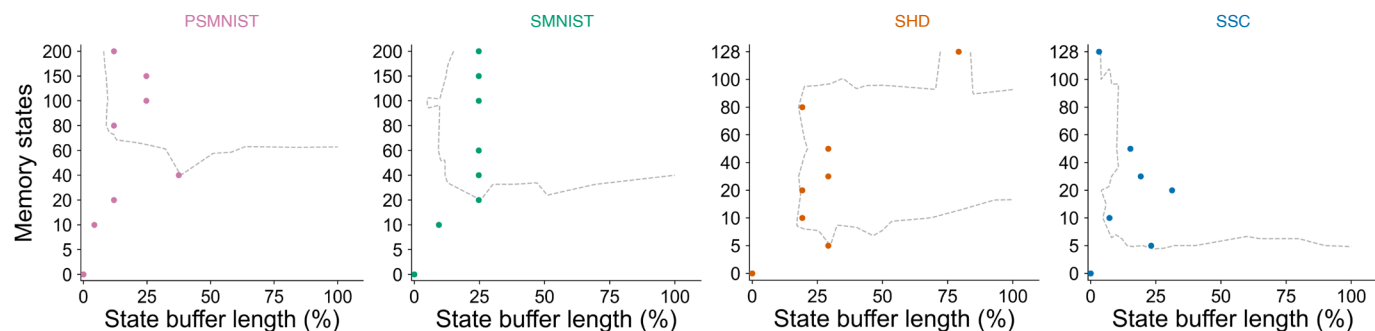
Extended Data Table 3 | Dimensions of tensors in the DMP-SNN at layer l

Symbol	Role	Dimension
$s^{l-1}[k]$	Presynaptic spikes	$\{0,1\}^M$
$u^l[k]$	Membrane potentials (postsynaptic)	R^N
$m^l[k]$	Slow memory state	R^d
$x^l[k]$	Memory input	R^1
$I^l[k]$	Input current	R^N
$I_m^l[k]$	Memory current	R^N
W_f	Feedforward ($s \rightarrow u$)	$R^{N \times M}$
W_m	Memory readout ($m \rightarrow u$)	$R^{N \times d}$
$\bar{A}$	Memory transition (state matrix)	$R^{d \times d}$
$\bar{B}$	Memory input map ($x \rightarrow m$)	$R^{d \times 1}$
W_x	Spike compression ($s \rightarrow x$)	$R^{1 \times M}$
b	Bias for x	R^1
W_r	Recurrent ($s \rightarrow u$)	$R^{N \times N}$
$\bar{d}^l$	Axonal delays (per output channel)	R^N

M is the presynaptic width, N the number of postsynaptic neurons, and d the memory dimension.

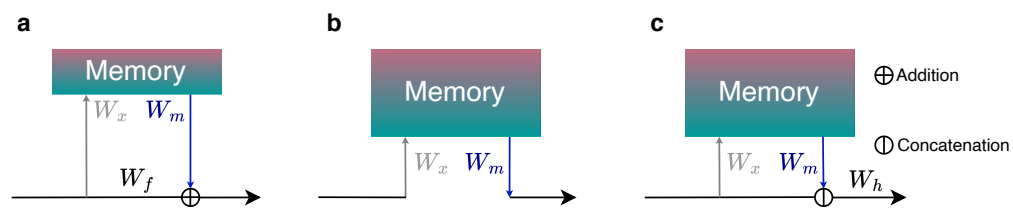
Extended Data Table 4 | Hyperparameter settings for different datasets

Parameter	SHD	SSC	S/PS-MNIST	Description
Input channels	140	140	1	Number of input units
Simulation Time Steps	100	250	784	Simulation Time Steps
Output neurons	20	35	10	Number of classes
Hidden Spiking Neurons	128	128	200	Size of hidden layer
Memory states	10	10	40	Dimension of memory states
State buffer length	40	40	300	Sliding window length theta
Number of Hidden Layers	2	1 and 2	2	Number of hidden layers
theta_u		10		Spiking threshold
f_x		ReLU		Non-linear function
bar d		learnable		Axonal delays



Extended Data Fig. 1 | Competitive accuracy across memory-state sizes and buffer lengths. For each memory-state size, we sweep the state-buffer length and report the competitive configurations (within 1.0% of the best for that size on SHD, and within 0.5% on the other tasks), for PS-MNIST, S-MNIST, SHD and

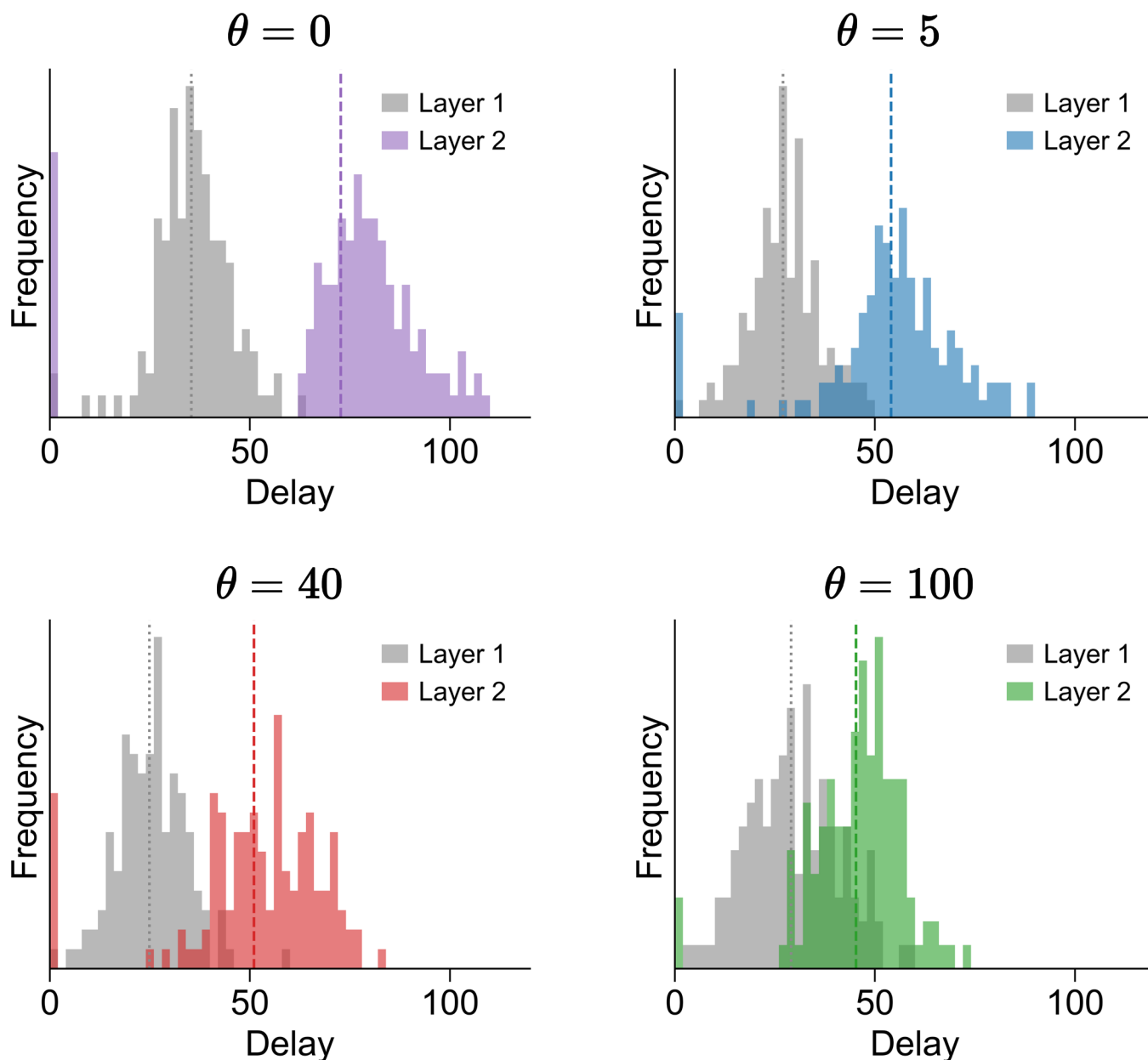
SSC. The grey dashed boundary marks the predefined competitive-performance threshold (defined in the text), highlighting that high performance is typically achieved with compact memory states and relatively short buffers.



Extended Data Fig. 2 | Architectural comparison and feedforward ablation.

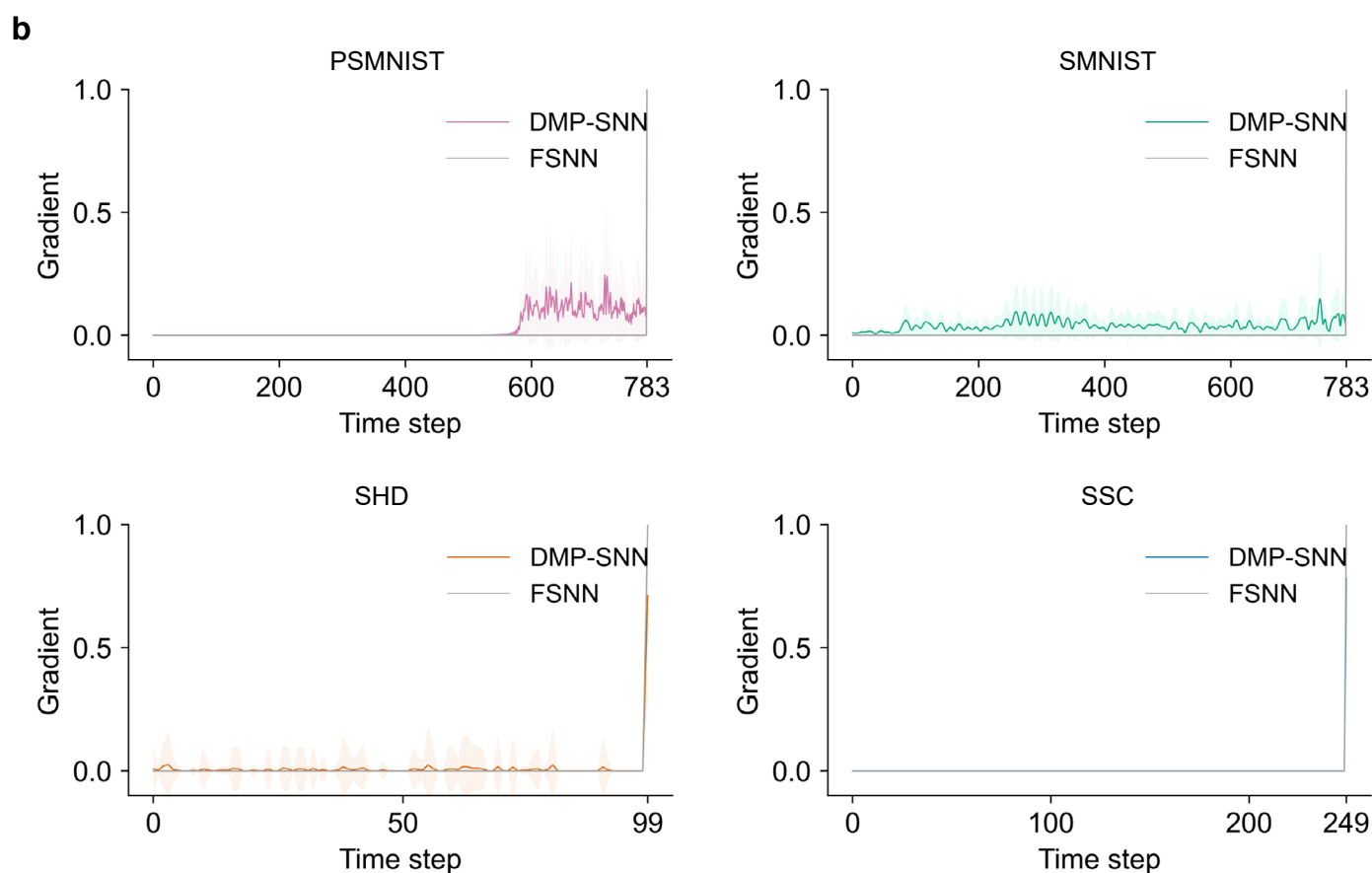
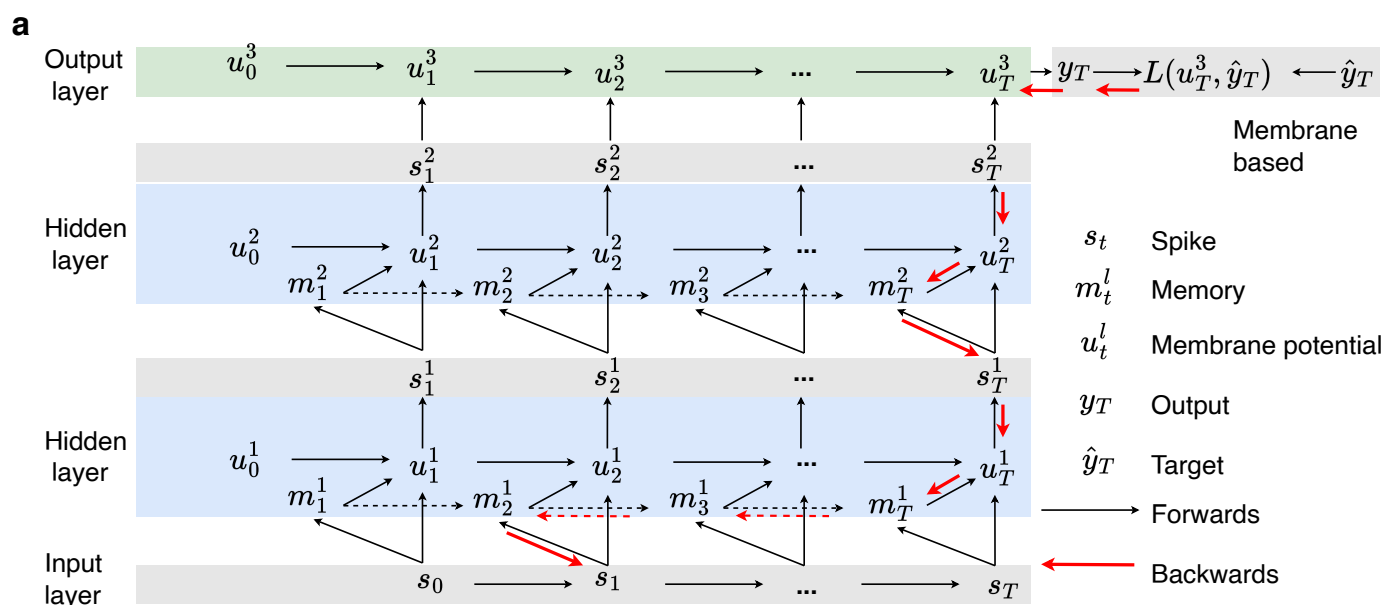
a, Proposed DMP-SNN with a direct feedforward pathway (W_f) alongside a shared slow state pathway. **b**, Feedforward ablation ($W_f = 0$), yielding a state-only model; performance collapses to chance across tasks, indicating that the slow state acts

as a contextual modulator. **c**, Representative spiking state-space model (Spiking SSM) that mixes input and state and applies an additional learned transformation (W_h), increasing model complexity and compute relative to the DMP design.



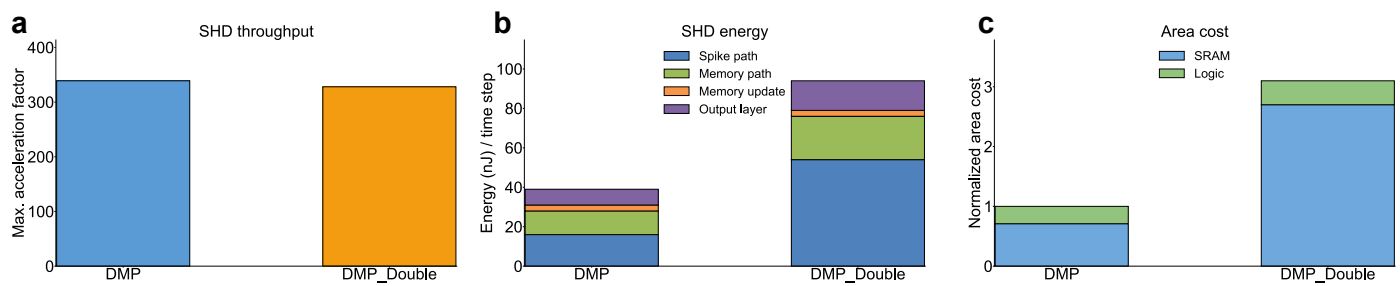
Extended Data Fig. 3 | Effect of longer state buffer length on axonal delay distributions. A longer state buffer length (θ) partially absorbs the temporal span that would otherwise be handled by longer axonal delays. The panels show

the learned delay distributions as θ increases while all other settings are held the same. As θ grows, the probability mass shifts toward shorter delays. The vertical line in the dashed line represents the average delay.



Extended Data Fig. 4 | Vanilla spiking neurons struggle with long-term credit assignment. To clearly assess long-range credit assignment, we adapt the strictly last-timestep supervision. We examined how gradients propagate through time in the dual-memory pathway spiking network (DMP-SNN) and feedforward SNN (FSNN). **a**, Computational graph unrolled through time for a spiking neuron during backpropagation through time (BPTT). Red arrows denote the slow gradient pathway visualised in **b**. **b**, Normalised gradient magnitude traced from

the terminal loss back to the first hidden layer for PS-MNIST, S-MNIST, SHD, and SSC. Solid lines show the mean and shaded regions indicate the s.e.m. across a random batch (batch size = 256). Under this strict supervision regime, the DMP-SNN maintains a broad temporal tail, reflecting stable long-range credit assignment through its slow memory pathway, whereas the feedforward baseline decays rapidly, relying solely on fast membrane dynamics.



Extended Data Fig. 5 | Hardware scaling analysis. Hardware scaling analysis when doubling the LIF neuron layer (denoted as DMP_Double). **(a)** Throughput is largely maintained with $4\times$ MAC and LIF logic, with a slight reduction due to decreased spike sparsity. **(b)** Energy breakdown across spike, memory, memory update, and output paths. Total energy increases by $2.4\times$ (sub-quadratic vs. expected $\sim 4\times$), with the spike path becoming dominant ($\approx 59\%$). **(c)** Area breakdown into SRAM and logic. Total area increases by $3.1\times$, with SRAM (weights and states) accounting for $\approx 87\%$ of the area. From the algorithmic results in Fig. 3(b), we observe that the memory state remains relatively compact, and increasing its size does not necessarily provide additional performance gains. This suggests that, under scaling, variations in the number of LIF neurons have a more direct impact on overall performance compared to scaling the memory state. Accordingly, on the hardware side, we have: (1) added a detailed cost breakdown of the major components shown in Fig. 4(a), and (2) scaled the LIF neuron count to 256 and reported the corresponding hardware metrics. We maintain a single-core design, as extending to a multi-core architecture would require network partitioning, mapping, and routing optimizations, which are beyond the scope of this work. To preserve hardware throughput under scaling, both the MAC array and LIF neuron logic are increased by $4\times$. As shown in Extended Data Fig. 5(a-c), we evaluate the scalability of throughput, energy, and area when doubling the size of the LIF neuron layer. The energy breakdown (Extended Data Fig. 5(b)) consists of four components corresponding to the data paths in Fig. 4(a): **• Spike path:** energy from LIF neuron logic and SRAM access. **• Memory path:** two parallel paths (blue and purple), accounting for MAC array computation and associated SRAM access. **• Memory update path.** **• Output layer.** In the area breakdown, SRAM includes storage for both weight parameters

and neuron states, while logic includes LIF neuron circuits, MAC units, and ReLU. **Throughput.** As shown in Extended Data Fig. 5(a), the throughput of DMP_Double is slightly lower than that of DMP, despite the $4\times$ increase in MAC and LIF logic. This is due to slightly reduced spike sparsity in the DMP_Double configuration. **Energy.** As shown in Extended Data Fig. 5(b), the total energy of DMP_Double increases by $2.4\times$ compared to DMP, which is significantly better than the expected $\sim 4\times$ increase (assuming quadratic growth in synaptic and MAC operations). The energy scaling factors for the spike path, memory path, memory update, and output layer are $3.4\times$, $1.8\times$, $1\times$, and $1.9\times$, respectively. The spike path becomes the dominant contributor (approximately 59% of total energy in DMP_Double), which aligns with expectations since only the number of spiking neurons is scaled, while the memory state size remains unchanged. The memory and output paths scale close to $2\times$ due to the increased number of spiking neurons. **Area.** The total silicon area of DMP_Double increases by $3.1\times$ compared to DMP. SRAM—particularly weight storage—gradually dominates the area, accounting for approximately 87% in DMP_Double. Although the computing logic (MAC and LIF) increases by $4\times$, its area growth remains modest relative to the rapid growth of SRAM area. **Conclusion.** The proposed DMP-SNN architecture decouples and parallelizes multiple computation paths, enabling:

- Good throughput scalability (throughput is largely maintained by scaling compute logic).
- Sub-quadratic energy growth, even though synaptic and MAC operations increase quadratically. Two key observations emerge:
 - The area increase of compute logic is negligible compared to SRAM growth.
 - The spike path increasingly dominates total energy under scaling.

These findings suggest that further exploration of sparse weight representations in DMP-SNN is a promising direction to reduce both energy and area costs.